

Anotación @Fetch: Para guiar las consultas

La anotación @Fetch en JPA es una extensión de **Hibernate** que permite especificar cómo se obtendrán las entidades relacionadas en una consulta. Esta anotación no es parte de JPA estándar, sino de Hibernate, por lo que solo funciona si tu proyecto está configurado para usar Hibernate como proveedor de persistencia.

¿Qué es @Fetch y cuándo usarla?

La anotación @Fetch de Hibernate se utiliza para ajustar el comportamiento de carga de las relaciones en las entidades, especialmente cuando necesitas controlar cómo se recuperan los datos relacionados en las relaciones de tipo @OneToMany o @ManyToMany.

Existen tres estrategias principales que puedes definir con @Fetch:

- `FetchMode.SELECT`: Carga los elementos relacionados mediante consultas SQL separadas.
- `FetchMode.JOIN`: Carga los elementos relacionados utilizando una consulta SQL con JOIN.
- `FetchMode.SUBSELECT`: Carga los elementos relacionados utilizando una subconsulta SELECT cuando se accede a ellos.

Cada estrategia tiene su caso de uso ideal dependiendo de la cantidad de datos y la complejidad de la relación.

Ejemplos Prácticos de @Fetch en Spring Boot con JPA

A continuación, veremos cómo aplicar @Fetch en una relación @OneToMany. Imaginemos que tenemos dos entidades: `Autor` y `Libro`, donde un autor puede tener varios libros.

Definición de entidades `Autor` y `Libro`

- `Autor.java`

```
import jakarta.persistence.*;
import org.hibernate.annotations.Fetch;
import org.hibernate.annotations.FetchMode;
import java.util.List;

@Entity
public class Autor {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String nombre;

    @OneToMany(mappedBy = "autor", fetch = FetchType.LAZY)
```

```

    @Fetch(FetchMode.JOIN) // Estrategia de carga
    private List<Libro> libros;

    // Getters y Setters
}

```

- Libro.java

```

import jakarta.persistence.*;

@Entity
public class Libro {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String titulo;

    @ManyToOne
    @JoinColumn(name = "autor_id")
    private Autor autor;

    // Getters y Setters
}

```

Estrategias de @Fetch explicadas con ejemplos

1. @Fetch(FetchMode.SELECT)

Esta estrategia carga cada entidad relacionada con una consulta SQL separada. Es ideal cuando tienes pocas entidades relacionadas y deseas evitar una consulta con JOIN pesada.

```

@OneToMany(mappedBy = "autor", fetch = FetchType.LAZY)
@Fetch(FetchMode.SELECT)
private List<Libro> libros;

```

Ejemplo de consultas generadas:

1. La primera consulta obtiene los datos de Autor.
2. Por cada Autor, Hibernate ejecuta una consulta adicional para obtener los Libro asociados.

```

SELECT * FROM Autor WHERE id = ?
SELECT * FROM Libro WHERE autor_id = ?
SELECT * FROM Libro WHERE autor_id = ?

```

2. @Fetch(FetchMode.JOIN)

Esta estrategia realiza una consulta JOIN única para obtener la entidad principal y sus entidades relacionadas. Es útil cuando se necesita cargar todas las entidades relacionadas al mismo tiempo

y se espera que haya una cantidad limitada de relaciones.

```
@OneToMany(mappedBy = "autor", fetch = FetchType.LAZY)
@Fetch(FetchMode.JOIN)
private List<Libro> libros;
```

Ejemplo de consulta generada:

1. La consulta realiza un JOIN entre Autor y Libro.

```
SELECT a.*, l.* FROM Autor a
LEFT JOIN Libro l ON a.id = l.autor_id
WHERE a.id = ?
```

> **Nota:** Esta estrategia es útil cuando no hay una gran cantidad de elementos en la relación. De lo contrario, el resultado podría multiplicarse innecesariamente debido a la cantidad de datos obtenidos en el JOIN.

3. @Fetch(FetchMode.SUBSELECT)

Esta estrategia utiliza una **subconsulta** para cargar todas las entidades relacionadas en una única consulta cuando se accede a ellas. Puede ser más eficiente que `FetchMode.SELECT` cuando se recuperan varias entidades relacionadas al mismo tiempo.

```
@OneToMany(mappedBy = "autor", fetch = FetchType.LAZY)
@Fetch(FetchMode.SUBSELECT)
private List<Libro> libros;
```

Ejemplo de consulta generada:

1. Hibernate ejecuta una consulta para recuperar los Autor.
2. Cuando se accede a los Libro de cada Autor, Hibernate ejecuta una subconsulta para obtener todos los libros relacionados.

```
SELECT * FROM Autor WHERE id IN (?, ?, ?)
SELECT * FROM Libro WHERE autor_id IN (1, 2, 3)
```

Comparación de los Modos de @Fetch

Estrategia	Descripción	Caso de Uso Ideal
<code>FetchMode.SELECT</code>	Ejecuta una consulta separada para cada entidad relacionada.	Relaciones pequeñas, o cuando se accede siempre.
<code>FetchMode.JOIN</code>	Realiza un JOIN para obtener la entidad principal y la relacionada en una consulta única.	Relaciones con pocas entidades.
<code>FetchMode.SUBSELECT</code>	Ejecuta una subconsulta para obtener las entidades relacionadas cuando se accede a ellas.	Relación grande que se recupera cuando se necesita.

Consideraciones Importantes

1. Rendimiento:

- Las estrategias de `@Fetch` afectan el rendimiento de las consultas. `FetchMode.JOIN` es adecuado para pocas relaciones, mientras que `FetchMode.SUBSELECT` puede ser mejor para relaciones más grandes.

2. N+1 Problem:

- El uso de `FetchMode.SELECT` puede conducir al problema de **N+1** si no se gestiona adecuadamente. Esto sucede cuando se ejecutan múltiples consultas adicionales para cada registro relacionado, aumentando exponencialmente el número de consultas.

3. Compatibilidad:

- `@Fetch` es una anotación específica de Hibernate y no es parte de JPA estándar. Si decides cambiar el proveedor de persistencia, las configuraciones de `@Fetch` no se aplicarán.

Conclusión

La anotación `@Fetch` en Hibernate te permite controlar el comportamiento de carga de las relaciones de una entidad. Elegir la estrategia correcta (`SELECT`, `JOIN`, `SUBSELECT`) depende del tamaño de las relaciones y de cómo se acceden.

- Ejemplo práctico de `@OneToMany` con `@Fetch` en una relación de entidades

Vamos a trabajar con las entidades 'Autor' y 'Libro', donde un **Autor** puede tener múltiples **Libros** (relación '@OneToMany'). Además, aplicaremos `@Fetch` para ajustar el modo de carga de los libros en función de nuestras necesidades.

Estructura del ejemplo

- 'Autor': Representa al autor de los libros. Cada autor puede tener múltiples libros asociados.
- 'Libro': Representa un libro que pertenece a un autor.

Configuración de las Entidades con '@OneToMany' y '@Fetch'

1. Entidad 'Autor'

En la entidad 'Autor', definimos una relación `@OneToMany` con 'Libro' y aplicamos `@Fetch` para controlar el comportamiento de carga de los libros.

```
import jakarta.persistence.*;
import org.hibernate.annotations.Fetch;
import org.hibernate.annotations.FetchMode;
import java.util.List;

@Entity
public class Autor {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
```

```

private String nombre;

@OneToMany(mappedBy = "autor", fetch = FetchType.LAZY)
@Fetch(FetchMode.JOIN) // Cambia esto a FetchType.SELECT o FetchType.SUBSELE
private List<Libro> libros;

// Getters y Setters
public Long getId() {
    return id;
}

public void setId(Long id) {
    this.id = id;
}

public String getNombre() {
    return nombre;
}

public void setNombre(String nombre) {
    this.nombre = nombre;
}

public List<Libro> getLibros() {
    return libros;
}

public void setLibros(List<Libro> libros) {
    this.libros = libros;
}
}

```

Explicación:

- **@OneToMany**: Declara la relación de un autor a múltiples libros.
- **@Fetch(FetchMode.JOIN)**: Usamos 'FetchMode.JOIN' para realizar una consulta 'JOIN' que recupere el autor y sus libros al mismo tiempo. Esta estrategia es útil cuando queremos cargar la lista de libros en una única consulta SQL.

> Puedes cambiar 'FetchMode.JOIN' a 'FetchMode.SELECT' o 'FetchMode.SUBSELECT' según la estrategia que prefieras.

2. Entidad 'Libro'

La entidad 'Libro' tiene una relación **@ManyToOne** con 'Autor', ya que cada libro tiene un único autor.

```

import jakarta.persistence.*;

@Entity
public class Libro {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String titulo;

    @ManyToOne
    @JoinColumn(name = "autor_id")
    private Autor autor;

    // Getters y Setters
    public Long getId() {
        return id;
    }

    public void setId(Long id) {
        this.id = id;
    }

    public String getTitulo() {
        return titulo;
    }

    public void setTitulo(String titulo) {
        this.titulo = titulo;
    }

    public Autor getAutor() {
        return autor;
    }

    public void setAutor(Autor autor) {
        this.autor = autor;
    }
}

```

Explicación:

- **@ManyToOne**: Indica que cada libro pertenece a un solo autor.
- **@JoinColumn(name = "autor_id")**: Define la columna de la base de datos que almacena la clave foránea para el autor en la tabla 'Libro'.

Uso de '@Fetch' en Diferentes Escenarios

1. 'FetchMode.JOIN'

Usar 'FetchMode.JOIN' en el ejemplo de 'Autor' generará una consulta 'JOIN' entre 'Autor' y 'Libro'. Esto es útil cuando esperamos un número reducido de relaciones y queremos optimizar la consulta para obtener los datos relacionados en una sola ejecución.

```
@OneToMany(mappedBy = "autor", fetch = FetchType.LAZY)
@Fetch (FetchMode.JOIN)
private List<Libro> libros;
```

Consulta SQL generada:

```
SELECT a.*, l.* FROM Autor a
LEFT JOIN Libro l ON a.id = l.autor_id
WHERE a.id = ?
```

Esta consulta carga el autor y sus libros en una sola consulta. Sin embargo, si hay muchos libros por autor, el rendimiento puede verse afectado.

2. 'FetchMode.SELECT'

Usar 'FetchMode.SELECT' genera una consulta adicional para cada autor al acceder a sus libros. Es ideal si no accedemos a la lista de libros frecuentemente.

```
@OneToMany(mappedBy = "autor", fetch = FetchType.LAZY)
@Fetch (FetchMode.SELECT)
private List<Libro> libros;
```

Consultas SQL generadas:

1. La consulta inicial para obtener el autor:

```
SELECT * FROM Autor WHERE id = ?
```

1. Una consulta separada para cada autor cuando accedemos a sus libros:

```
SELECT * FROM Libro WHERE autor_id = ?
```

> **Nota:** 'FetchMode.SELECT' puede causar el problema de N+1 si tenemos una lista grande de autores y accedemos a los libros de cada uno, ya que se ejecutará una consulta adicional por cada autor.

3. 'FetchMode.SUBSELECT'

Usar 'FetchMode.SUBSELECT' es una buena opción cuando se cargan múltiples entidades principales al mismo tiempo. Hibernate ejecutará una sola subconsulta para cargar todas las entidades relacionadas cuando accedamos a ellas.

```
@OneToMany(mappedBy = "autor", fetch = FetchType.LAZY)
@Fetch (FetchMode.SUBSELECT)
private List<Libro> libros;
```

Consultas SQL generadas:

1. La consulta para cargar varios autores:

```
SELECT * FROM Autor WHERE id IN (?, ?, ?)
```

1. Una subconsulta que recupera todos los libros para los autores cargados:

```
SELECT * FROM Libro WHERE autor_id IN (1, 2, 3)
```

Esta estrategia es útil cuando necesitas cargar todas las relaciones en una sola consulta, pero solo después de haber cargado las entidades principales.

Ejemplo de Prueba en un Servicio o Controlador

Método en el Servicio 'AutorService'

```
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

@Service
public class AutorService {

    @Autowired
    private AutorRepository autorRepository;

    @Transactional(readOnly = true)
    public Autor obtenerAutorConLibros(Long autorId) {
        return autorRepository.findById(autorId).orElseThrow(()
            -> new RuntimeException("Autor no encontrado"));
    }
}
```

Controlador 'AutorController'

```
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/autores")
public class AutorController {

    @Autowired
    private AutorService autorService;

    @GetMapping("/{id}/libros")
    public Autor obtenerAutorConLibros(@PathVariable Long id) {
        return autorService.obtenerAutorConLibros(id);
    }
}
```

Cuando accedes a '/autores/{id}/libros', verás el comportamiento de '@Fetch' en acción. Según la estrategia ('JOIN', 'SELECT', o 'SUBSELECT') configurada en la entidad 'Autor', Hibernate ejecutará las consultas de manera diferente.

Resumen de Estrategias de '@Fetch' en '@OneToMany'

Estrategia de '@Fetch'	Descripción	Escenario Ideal
'FetchMode.JOIN'	Consulta única 'JOIN' para cargar el autor y sus libros.	Relación pequeña, acceso frecuente.
'FetchMode.SELECT'	Una consulta para cada relación cuando se accede a ella.	Relación ocasionalmente accedida, para evitar grandes JOIN.
'FetchMode.SUBSELECT'	Usa una subconsulta para cargar todas las relaciones a la vez.	Relación grande con carga diferida.

Cada estrategia tiene sus ventajas y desventajas, y su uso depende del contexto de la aplicación y la cantidad de datos relacionados. ¡Espero que este ejemplo te haya ayudado a entender cómo usar '@Fetch' en tus relaciones '@OneToMany' en JPA y Spring Boot!